Modern Python Tooling in the AI Coding Era

CommunityDays KC 2026

Micah Melling

Data Science / Technology Executive

UMKC Analytics Board Member

Former Adjunct Professor at Park University

Masters in Data Science from Rockhurst

Goal:

Talk about modern Python tooling and why grounded knowledge still matters in the age of AI coding assistance.

Development standards prevent the “wild west”, help us manage tech debt, and allow for better code reviews.

These items have perhaps never been more important.

Grounded human knowledge of software is still vitally important!

Agenda

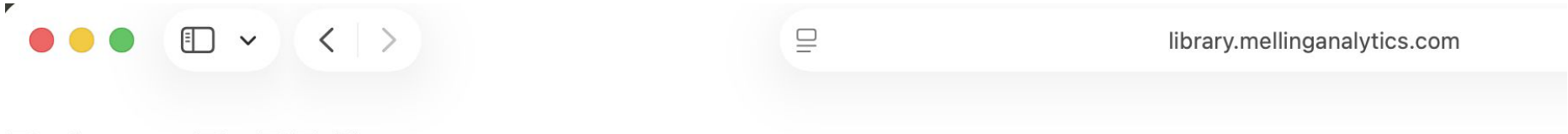
Enforcing development standards for human and LLM code.

Reviewing modern Python tooling options.

Enforcing Development Standards

Building a Private Python Package

- If we have significant amounts of shared code, we can create a private Python package.



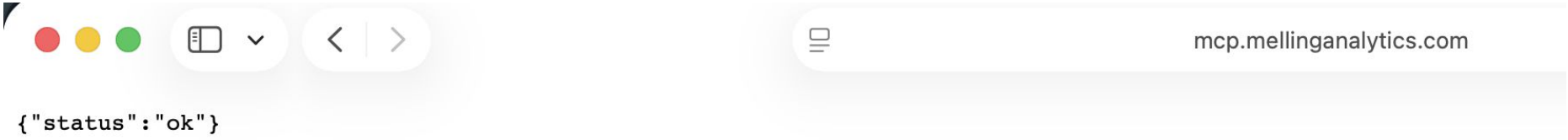
Private PyPI Server

Available Packages:

- awesomeutils

Model Context Protocol - MCP

- Specifically for coding agents, we can build our own remote MCP server, which is theoretically a promising option to standardize coding practices.
- This protocol allows us to give our coding agents access to functions and data we have approved.
- MCP is not a panacea because the agents still have to use the tools properly.



{"status":"ok"}

Verify customer MCP server ···

Verify the customer MCP server mellinganalytics works

Worked for 2m 23s >

Verified.

The live `mellinganalytics` MCP server at `https://mcp.mellinganalytics.com` works over its SSE flow:

- `GET /sse` returns a session endpoint like `/messages?session_id=...`
- `initialize` succeeds and reports server `mcp-poc` version `1.27.1`
- `tools/list` returns two tools:
  - `get_weather`
  - `calculate_hash`
- I called `calculate_hash` with `hello` and got the expected SHA-256:
  `2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824`

Use the Mellinganalytics MCP server to create a hash for "kansascity"

Worked for 27s >

Using the Mellinganalytics MCP server, the SHA-256 hash for `kansascity` is:

`b347a95eec34f887b35c64270174f8a42c630ce5cf2a87b71945fa32d040b207`

Project Templates with cookiecutter

- We often don’t want to start from a blank repo but from a template based on the type of project. We can accomplish this with cookiecutter.
  - Directory structure
  - Environment set up files
  - Security protocols
  - Development standard (claude.md)

Code Standards with pre-commit

- We can deterministically format code and enforce coding standards by running pre-commit hooks.
- Here is a helpful example.

Making Decisions on Development Preferences

Moving From Pandas → Polars

- Written in Rust, Polars is much more performant that Pandas.
- The syntax is more verbose, but the switch is worthwhile.

When Streamlit Won’t Cut It

Streamlit is great but has some challenges.

- Top-to-bottom execution model can be limited for complex apps
- Authentication frameworks are messy
- Dealing with lots of data can be challenging
- Frontend customization is clunky

What are my alternatives? mesop, dash, anvil, vibe-coded frontend

FastAPI, Flask, and Everything Else

- Flask is still good!
- FastAPI has become the new default.
- Many other specialized frameworks exist (tornado, quart).

Hyperparameter Optimization Library

- RIP hyperopt.
- scikit-learn optimization routines are still pretty basic.
- Are optuna and raytune the modern options?

Questions?

micahmelling@gmail.com